Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №4

По дисциплине «Обработка изображений в интеллектуальных системах»

Тема: «Трекинг множественных объектов»

Выполнил:

Студент 4 курса

Группы ИИ-26

Петручик Д.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов

Ход работы:

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;

2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);

3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;

4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;

5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Код программы:

```
import os
import time
import random
import shutil
import subprocess
from pathlib import Path
from getpass import getpass
from collections import defaultdict

import cv2
import yaml
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import torch
import ultralytics

from roboflow import Roboflow
from ultralytics import YOLO
from google.colab import files
from IPython.display import Video, display

SEED = 42
EPOCHS = 20
IMGSZ = 640
```

```
BATCH = 8
DEVICE = 0 if torch.cuda.is_available() else "cpu"

START = 0
DURATION = 20

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)

ROOT = Path("/content/lab4_cats")
ROOT.mkdir(exist_ok=True)

RUN = ROOT / time.strftime("run_%Y%m%d_%H%M%S")
RUN.mkdir(exist_ok=False)

WEIGHTS = ROOT / "cats_yolo12s_best.pt"

print("Вариант 10: YOLO12s, коты")
print("Устройство:", DEVICE)
print("Результаты:", RUN)

if not WEIGHTS.exists():
    api_key = os.getenv("ROBOFLOW_API_KEY") or getpass(
        "Введите API-ключ Roboflow: "
    )

    rf = Roboflow(api_key=api_key)

    dataset = (
        rf.workspace("mohamed-traore-2ekkp")
        .project("cats-n9b87")
        .version(3)
        .download(
            "yolov8",
            location=str(ROOT / "dataset")
        )
    )

    dataset_dir = Path(dataset.location).resolve()
    original_yaml = dataset_dir / "data.yaml"

    data_cfg = yaml.safe_load(
        original_yaml.read_text(encoding="utf-8")
    )

    data_cfg["path"] = str(dataset_dir)

    for key, split in [
        ("train", "train"),
        ("val", "valid"),
        ("test", "test"),
    ]:
        folder = dataset_dir / split / "images"
```

```
    if not folder.exists() or not any(
        p.suffix.lower() in {".jpg", ".jpeg", ".png", ".bmp", ".webp"}
        for p in folder.iterdir()
    ):
        raise RuntimeError(f"Нет изображений: {folder}")

    data_cfg[key] = str(folder)

yaml_path = RUN / "data.yaml"

yaml_path.write_text(
    yaml.safe_dump(data_cfg, allow_unicode=True),
    encoding="utf-8",
)

print("Классы датасета:", data_cfg["names"])

model = YOLO("yolo12s.pt")

model.train(
    data=str(yaml_path),
    epochs=EPOCHS,
    imgsz=IMGSZ,
    batch=BATCH,
    device=DEVICE,
    workers=0,
    seed=SEED,
    project=str(RUN),
    name="training",
    plots=True,
)

trained_weights = (
    Path(model.trainer.save_dir) / "weights" / "best.pt"
)

shutil.copy2(trained_weights, WEIGHTS)

metrics = YOLO(str(WEIGHTS)).val(
    data=str(yaml_path),
    split="test",
    imgsz=IMGSZ,
    batch=BATCH,
    device=DEVICE,
    workers=0,
    project=str(RUN),
    name="test_evaluation",
)

detector_metrics = pd.DataFrame([{
    "Precision": float(metrics.box.mp),
    "Recall": float(metrics.box.mr),
    "mAP50": float(metrics.box.map50),
```

```
            "mAP50-95": float(metrics.box.map),
        }])

        detector_metrics.to_csv(
            RUN / "detector_metrics.csv",
            index=False,
        )

        display(detector_metrics)

else:
    print("Используются ранее обученные веса:", WEIGHTS)

check_model = YOLO(str(WEIGHTS))

names = check_model.names
items = names.items() if isinstance(names, dict) else enumerate(names)

CAT_IDS = [
    int(class_id)
    for class_id, name in items
    if str(name).strip().lower()
    in {"cat", "cats", "кот", "коты", "кошка", "кошки"}
]

if not CAT_IDS:
    raise RuntimeError(
        f"Не найден класс котов. Имена классов модели: {names}"
    )

del check_model

print("ID классов котов:", CAT_IDS)

print("Загрузи один ролик с несколькими котами.")
uploaded = files.upload()

if len(uploaded) != 1:
    raise RuntimeError("Нужно загрузить ровно один видеофайл.")

uploaded_name = next(iter(uploaded))

source = Path(uploaded_name).resolve()

source_url = input(
    "Ссылка на источник ролика для отчёта "
    "(если видео своё — напиши «собственное»): "
).strip()

(RUN / "video_source.txt").write_text(
    f"Файл: {uploaded_name}\n"
    f"Источник: {source_url or 'Не указан'}\n"
    f"Начало: {START} с\n"
    f"Длительность фрагмента: {DURATION} с\n",
```

```
    encoding="utf-8",
)

VIDEO = RUN / "video.mp4"

subprocess.run(
    [
        "ffmpeg", "-y", "-loglevel", "error",
        "-i", str(source),
        "-ss", str(START),
        "-t", str(DURATION),
        "-an",
        "-c:v", "libx264",
        "-pix_fmt", "yuv420p",
        str(VIDEO),
    ],
    check=True,
)

cap = cv2.VideoCapture(str(VIDEO))
ok, _ = cap.read()

video_fps = cap.get(cv2.CAP_PROP_FPS)
video_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
video_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
video_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

cap.release()

if not ok:
    raise RuntimeError("Видео не читается или выбран пустой фрагмент.")

print(
    f"Кадров: {video_frames}, FPS: {video_fps:.2f}, "
    f"размер: {video_width} × {video_height}"
)

TRACKERS_DIR = (
    Path(ultralytics.__file__).resolve().parent
    / "cfg" / "trackers"
)

CONFIGS = RUN / "trackers"
CONFIGS.mkdir()

for name in ("botsort.yaml", "bytetrack.yaml"):
    cfg = yaml.safe_load(
        (TRACKERS_DIR / name).read_text(encoding="utf-8")
    )

    cfg.update(
        track_high_thresh=0.25,
        track_low_thresh=0.1,
        new_track_thresh=0.25,
```

```
            track_buffer=30,
            match_thresh=0.8,
            fuse_score=True,
        )

        if name == "botsort.yaml":
            cfg.update(
                gmc_method="sparseOptFlow",
                with_reid=False,
            )

        (CONFIGS / name).write_text(
            yaml.safe_dump(cfg, sort_keys=False),
            encoding="utf-8",
        )


def make_cfg(base_yaml, out_name, changes):
    cfg = yaml.safe_load(
        Path(base_yaml).read_text(encoding="utf-8")
    )
    cfg.update(changes)

    out = CONFIGS / out_name
    out.write_text(
        yaml.safe_dump(cfg, sort_keys=False),
        encoding="utf-8",
    )
    return out

CONF = 0.1

RESULTS = []
ALL_TRACKS = {}
OUTPUT_DIRS = {}


def run_tracking(tracker_cfg, label, folder, conf=CONF):
    model = YOLO(str(WEIGHTS))

    tracks = defaultdict(list)
    records = []

    n_frames = 0
    n_dets = 0
    max_objects = 0
    frames_with_tracks = 0

    started = time.perf_counter()

    stream = model.track(
        source=str(VIDEO),
        tracker=str(tracker_cfg),
        classes=CAT_IDS,
```

```
        conf=conf,
        imgsz=IMGSZ,
        device=DEVICE,
        stream=True,
        save=True,
        project=str(RUN / "tracking"),
        name=folder,
        exist_ok=False,
        verbose=False,
    )

    for frame_number, result in enumerate(stream):
        n_frames += 1

        boxes = result.boxes

        if boxes is None or boxes.id is None:
            continue

        ids = boxes.id.int().cpu().tolist()
        coordinates = boxes.xyxy.cpu().tolist()
        confidences = boxes.conf.cpu().tolist()

        n_dets += len(ids)
        max_objects = max(max_objects, len(ids))
        frames_with_tracks += bool(ids)

        for track_id, xyxy, confidence in zip(
            ids, coordinates, confidences
        ):
            tracks[track_id].append(frame_number)

            records.append({
                "frame": frame_number,
                "time_s": frame_number / video_fps,
                "track_id": track_id,
                "confidence": confidence,
                "x1": xyxy[0],
                "y1": xyxy[1],
                "x2": xyxy[2],
                "y2": xyxy[3],
            })

    elapsed = time.perf_counter() - started

    if n_frames == 0:
        raise RuntimeError(f"Не обработаны кадры: {label}")

    lengths = np.array(
        [len(frames) for frames in tracks.values()],
        dtype=int,
    )

    output_dir = Path(model.predictor.save_dir)
```

```
    OUTPUT_DIRS[label] = output_dir

    pd.DataFrame(
        records,
        columns=[
            "frame", "time_s", "track_id", "confidence",
            "x1", "y1", "x2", "y2",
        ],
    ).to_csv(output_dir / "tracks.csv", index=False)

    stats = {
        "эксперимент": label,
        "кадров": n_frames,
        "уникальных ID": len(tracks),
        "средняя длина трека (кадров)": (
            round(float(lengths.mean()), 2)
            if lengths.size else 0.0
        ),
        "коротких треков (<15 кадров)": int(
            (lengths < 15).sum()
        ),
        "объектов на кадр": round(n_dets / n_frames, 3),
        "максимум одновременно": max_objects,
        "доля кадров с треками": round(
            frames_with_tracks / n_frames, 3
        ),
        "FPS полного прохода": round(
            n_frames / max(elapsed, 1e-9), 2
        ),
    }

    return stats, dict(tracks)

BYTE = CONFIGS / "bytetrack.yaml"
BOT = CONFIGS / "botsort.yaml"

EXPERIMENTS = [
    ("BoT-SORT: база", "bot_base", BOT, {}, CONF),
    ("ByteTrack: база", "byte_base", BYTE, {}, CONF),

    ("ByteTrack: buffer=10", "byte_buf10",
     BYTE, {"track_buffer": 10}, CONF),

    ("ByteTrack: buffer=90", "byte_buf90",
     BYTE, {"track_buffer": 90}, CONF),

    ("ByteTrack: high=0.5", "byte_high05",
     BYTE, {"track_high_thresh": 0.5}, CONF),

    ("ByteTrack: match=0.5", "byte_match05",
     BYTE, {"match_thresh": 0.5}, CONF),

    ("ByteTrack: match=0.95", "byte_match095",
     BYTE, {"match_thresh": 0.95}, CONF),
```

```
    ("ByteTrack: conf=0.25", "byte_conf025",
     BYTE, {}, 0.25),

    ("BoT-SORT: buffer=10", "bot_buf10",
     BOT, {"track_buffer": 10}, CONF),

    ("BoT-SORT: buffer=90", "bot_buf90",
     BOT, {"track_buffer": 90}, CONF),

    ("BoT-SORT: без GMC", "bot_nogmc",
     BOT, {"gmc_method": "none"}, CONF),

    ("BoT-SORT: conf=0.25", "bot_conf025",
     BOT, {}, 0.25),
]

for label, folder, base, changes, conf in EXPERIMENTS:
    config_path = make_cfg(
        base, f"{folder}.yaml", changes
    )

    print("Запуск:", label)

    stats, tracks = run_tracking(
        config_path, label, folder, conf
    )

    RESULTS.append(stats)
    ALL_TRACKS[label] = tracks

    pd.DataFrame(RESULTS).to_csv(
        RUN / "comparison.csv",
        index=False,
        encoding="utf-8-sig",
    )

    print("Готово:", label)

df = pd.DataFrame(RESULTS)
display(df)

columns = [
    "уникальных ID",
    "средняя длина трека (кадров)",
    "коротких треков (<15 кадров)",
]

fig, axes = plt.subplots(
    1, 3, figsize=(19, 8), sharey=True
)

for ax, column in zip(axes, columns):
    ax.barh(df["эксперимент"], df[column])
```

```
    ax.set_title(column)
    ax.grid(alpha=0.3, axis="x")

axes[0].invert_yaxis()

plt.tight_layout()
plt.savefig(RUN / "comparison.png", dpi=160)
plt.show()

labels = ["BoT-SORT: база", "ByteTrack: база"]

fig, axes = plt.subplots(
    1, 2, figsize=(16, 7), sharex=True
)

for ax, label in zip(axes, labels):
    items = sorted(
        ALL_TRACKS[label].items(),
        key=lambda item: item[1][0],
    )

    for row, (track_id, frames) in enumerate(items):

        ax.scatter(
            frames,
            np.full(len(frames), row),
            marker="|",
            s=12,
        )

    ax.set_title(f"{label}\nКоличество ID: {len(items)}")
    ax.set_xlabel("Номер кадра")
    ax.set_ylabel("Трек по порядку появления")
    ax.grid(alpha=0.3)

plt.tight_layout()
plt.savefig(RUN / "track_visibility.png", dpi=160)
plt.show()

def show_tracked(label):
    folder = OUTPUT_DIRS[label]

    videos = sorted(
        p for p in folder.rglob("*")
        if p.suffix.lower() in {".avi", ".mp4"}
        and not p.name.endswith("_preview.mp4")
    )

    if not videos:
        raise FileNotFoundError(
            f"Нет выходного видео в {folder}"
        )

    output = folder / "result_preview.mp4"
```

```
    subprocess.run(
        [
            "ffmpeg", "-y", "-loglevel", "error",
            "-i", str(videos[0]),
            "-an",
            "-c:v", "libx264",
            "-pix_fmt", "yuv420p",
            "-crf", "26",
            str(output),
        ],
        check=True,
    )

    display(Video(str(output), embed=True, width=800))


show_tracked("ByteTrack: база")
show_tracked("BoT-SORT: база")


report = [
    "# Лабораторная работа №4",
    "",
    "## Вариант 10",
    "Детектор YOLO12s. Объекты: коты.",
    "",
    "## Цель",
    "Исследовать отслеживание нескольких объектов "
    "алгоритмами ByteTrack и BoT-SORT.",
    "",
    "## Подготовка",
    "Детектор обучается внутри этой работы на датасете "
    "cats-n9b87 версии 3. При повторном запуске в той же "
    "среде используются сохранённые веса.",
    f"Эпох при новом обучении: {EPOCHS}.",
    f"Размер входа: {IMGSZ}. Устройство: {DEVICE}.",
    f"Источник видео: {source_url or 'Не указан'}.",
    f"Обработано кадров фрагмента: {video_frames}.",
    "",
    "## Методика",
    "Все эксперименты используют одинаковые веса и видео. "
    "Для каждого эксперимента создаётся новая модель. "
    "В каждом опыте изменяется один параметр относительно "
    "базовой конфигурации соответствующего алгоритма.",
    "",
    "## Результаты",
    "| Эксперимент | ID | Средняя длина | Короткие треки | FPS |",
    "|---|---:|---:|---:|---:|",
]

for row in RESULTS:
    report.append(
        f"| {row['эксперимент']} "
```

```
            f"| {row['уникальных ID']} "
            f"| {row['средняя длина трека (кадров)']} "
            f"| {row['коротких треков (<15 кадров)']} "
            f"| {row['FPS полного прохода']} |"
        )

report += [
    "",
    "![Сравнение](comparison.png)",
    "",
    "![Видимость треков](track_visibility.png)",
    "",
    "## Интерпретация",
    "track_buffer определяет время сохранения потерянного "
    "трека. Увеличение помогает переживать перекрытия, но "
    "может повысить риск ошибочного сопоставления.",
    "",
    "track_high_thresh задаёт порог сильных детекций "
    "первого этапа. При повышении порога их становится меньше.",
    "",
    "Уменьшение match_thresh делает ограничение по стоимости "
    "сопоставления строже.",
    "",
    "gmc_method управляет компенсацией движения камеры. "
    "В базовом BoT-SORT включён sparseOptFlow; ReID выключен.",
    "",
    "Повышение conf до 0.25 отсекает слабые детекции до "
    "передачи трекеру и ограничивает второй этап ByteTrack.",
    "",
    "## Ограничения",
    "Уникальные ID не равны числу котов. "
    "Меньшее число ID само по себе не доказывает лучшее качество.",
    "Средняя длина — количество кадров с наблюдением ID, "
    "а не интервал между первым и последним появлением.",
    "Без эталонной покадровой разметки не вычислялись "
    "IDF1, MOTA и число смен идентичности.",
    "FPS включает обработку, отрисовку, запись видео и прогрев.",
    "",
    "## Визуальный анализ перед сдачей",
    "Просмотрите сохранённые видео и добавьте наблюдения "
    "с номерами кадров: смены ID, пропуски, ложные рамки "
    "и сохранение ID при перекрытии котов.",
]

if df["максимум одновременно"].max() < 2:
    report += [
        "",
        "ВНИМАНИЕ: одновременно не обнаружены два трека. "
        "Для демонстрации множественного трекинга требуется "
        "другой ролик или улучшение детектора.",
    ]

(RUN / "REPORT.md").write_text(
    "\n".join(report),
```

```
    encoding="utf-8",
)

(RUN / "versions.txt").write_text(
    f"ultralytics={ultralytics.__version__}\n"
    f"torch={torch.__version__}\n",
    encoding="utf-8",
)

shutil.copy2(WEIGHTS, RUN / "cats_yolo12s_best.pt")

archive = shutil.make_archive(
    str(RUN),
    "zip",
    root_dir=str(RUN),
)

print("Работа завершена.")
print("Отчёт:", RUN / "REPORT.md")
print("Архив:", archive)

files.download(archive)
```

Результат тестирования:

```
Creating new Ultralytics Settings v0.0.8 file ✅
View Ultralytics Settings with 'yolo settings' or at
'/root/.config/Ultralytics/settings.json'
Update Settings with 'yolo settings key=value', i.e. 'yolo settings
runs_dir=path/to/dir'. For help see https://docs.ultralytics.com/usage/settings
Вариант 10: YOLO12s, коты
Устройство: 0
Результаты: /content/lab4_cats/run_20261101_203551
Введите API-ключ Roboflow: ·····
loading Roboflow workspace...
loading Roboflow project...
Downloading Dataset Version Zip in /content/lab4_cats/dataset to yolov8::
100%|██████████| 61862/61862 [00:05<00:00, 11420.63it/s]


Extracting Dataset Version Zip to /content/lab4_cats/dataset in yolov8::
100%|██████████| 5574/5574 [00:00<00:00, 9193.75it/s]
Классы датасета: ['cat']
Downloading
https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo12s.pt to
'yolo12s.pt': 100% ━━━━━━━━━━━━━━━━━━━━ 18.1MB 64.2MB/s 0.3s
Ultralytics 8.4.197 🚀 Python-3.13.16 torch-2.11.0+cu130 CUDA:0 (Tesla T4,
14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False,
auto_augment=randaugment, batch=8, bgr=0.0, box=7.5, cache=False, cfg=None,
channels_last=None, classes=None, close_mosaic=10, cls=0.5, cls_pw=0.0,
cls_remap=True, compile=False, conf=None, copy_paste=0.0, copy_paste_mode=flip,
cos_lr=False, cutmix=0.0,
data=/content/lab4_cats/run_20261101_203551/data.yaml, degrees=0.0,
deterministic=True, device=0, dfl=1.5, dgrad=0.5, dis=6.0, distill_model=None,
dlam=1.0, dlog=1.0, dnn=False, dropout=0.0, dynamic=False, embed=None,
epochs=20, erasing=0.4, exist_ok=False, fliplr=0.5, flipud=0.0,
format=torchscript, fraction=1.0, freeze=None, hsv_h=0.015, hsv_s=0.7,
hsv_v=0.4, imgsz=640, iou=0.7, kobj=1.0, line_width=None, lr0=0.01, lrf=0.01,
```

```
mask_ratio=4, max_det=300, mixup=0.0, mode=train, model=yolo12s.pt,
momentum=0.937, mosaic=1.0, multi_scale=0.0, name=training, nbs=64, nms=None,
opset=None, optimize=False, optimizer=auto, overlap_mask=True, patience=100,
perspective=0.0, plots=True, pose=12.0, pretrained=True, profile=False,
project=/content/lab4_cats/run_20261101_203551, quantize=None, rect=False,
resume=False, retina_masks=False, rle=1.0, save=True, save_conf=False,
save_crop=False, save_dir=/content/lab4_cats/run_20261101_203551/training,
save_frames=False, save_json=False, save_period=-1, save_txt=False, scale=0.5,
seed=42, shear=0.0, show=False, show_boxes=True, show_conf=True,
show_labels=True, simplify=True, single_cls=False, source=None, split=val,
stream_buffer=False, task=detect, time=None, tracker=tracktrack.yaml,
translate=0.1, val=True, verbose=True, vid_stride=1, visualize=False,
warmup_bias_lr=0.1, warmup_epochs=3.0, warmup_momentum=0.8,
weight_decay=0.0005, workers=0, workspace=None
Downloading https://ultralytics.com/assets/Arial.ttf to
'/root/.config/Ultralytics/Arial.ttf': 100% ━━━━━━━━━━━━━━━━━━━━ 755.1KB
132.2MB/s 0.0s
Overriding model.yaml nc=80 with nc=1

                   from  n    params  module
arguments
  0                  -1  1       928  ultralytics.nn.modules.conv.Conv
[3, 32, 3, 2]
  1                  -1  1     18560  ultralytics.nn.modules.conv.Conv
[32, 64, 3, 2]
  2                  -1  1     26080  ultralytics.nn.modules.block.C3k2
[64, 128, 1, False, 0.25]
  3                  -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
  4                  -1  1    103360  ultralytics.nn.modules.block.C3k2
[128, 256, 1, False, 0.25]
  5                  -1  1    590336  ultralytics.nn.modules.conv.Conv
[256, 256, 3, 2]
  6                  -1  2    689408  ultralytics.nn.modules.block.A2C2f
[256, 256, 2, True, 4]
  7                  -1  1   1180672  ultralytics.nn.modules.conv.Conv
[256, 512, 3, 2]
  8                  -1  2   2689536  ultralytics.nn.modules.block.A2C2f
[512, 512, 2, True, 1]
  9                  -1  1         0  torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']
 10             [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 11                 -1  1    345856  ultralytics.nn.modules.block.A2C2f
[768, 256, 1, False, -1]
 12                 -1  1         0  torch.nn.modules.upsampling.Upsample
[None, 2, 'nearest']
 13             [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 14                 -1  1     95104  ultralytics.nn.modules.block.A2C2f
[512, 128, 1, False, -1]
 15                 -1  1    147712  ultralytics.nn.modules.conv.Conv
[128, 128, 3, 2]
 16            [-1, 11]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 17                 -1  1    296704  ultralytics.nn.modules.block.A2C2f
[384, 256, 1, False, -1]
 18                 -1  1    590336  ultralytics.nn.modules.conv.Conv
[256, 256, 3, 2]
 19             [-1, 8]  1         0  ultralytics.nn.modules.conv.Concat
[1]
 20                 -1  1   1511424  ultralytics.nn.modules.block.C3k2
[768, 512, 1, True]
 21        [14, 17, 20]  1    819795  ultralytics.nn.modules.head.Detect
[1, 16, None, [128, 256, 512]]
YOLOv12s summary: 272 layers, 9,253,523 parameters, 9,253,507 gradients, 23.5
GFLOPs
```

```
Remapped 1/1 cls head rows from pretrained weights by class name
Transferred 691/691 items from pretrained weights
Freezing layer 'model.21.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: checks passed ✓
train: Fast image access ✓ (ping: 0.0±0.0 ms, read: 792.9±386.4 MB/s, size:
19.9 KB)
train: Scanning /content/lab4_cats/dataset/train/labels... 2433 images, 0
backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━━━━━━━━━ 2433/2433 2.0Kit/s 1.2s
train: New cache created: /content/lab4_cats/dataset/train/labels.cache
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8,
8))
val: Fast image access ✓ (ping: 0.1±0.3 ms, read: 758.0±336.4 MB/s, size: 20.7
KB)
val: Scanning /content/lab4_cats/dataset/valid/labels... 232 images, 0
backgrounds, 0 corrupt: 100% ━━━━━━━━━━━━━━━━━━━━ 232/232 1.9Kit/s 0.1s
val: New cache created: /content/lab4_cats/dataset/valid/labels.cache
optimizer: 'optimizer=auto' found, ignoring 'lr0=0.01' and determining best
'optimizer' and 'lr0' automatically...
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 113
weight(decay=0.0), 120 weight(decay=0.0005), 119 bias(decay=0.0)
Plotting labels to
/content/lab4_cats/run_20261101_203551/training/labels.jpg...
Using 2433 train, 232 val images for fraction=1.0 at imgsz=640
Using 0 dataloader workers
Logging results to /content/lab4_cats/run_20261101_203551/training
Starting training for 20 epochs...

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       1/20      3.21G      1.214      1.132      1.783          4        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 3.6it/s 1:25
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 3.3it/s 4.6s
                   all        232        235      0.515      0.591      0.529
0.177

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       2/20      3.85G      1.244      1.014      1.812          2        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.0it/s 1:16
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.4it/s 3.4s
                   all        232        235      0.771      0.719      0.645
0.26

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       3/20      3.85G      1.189     0.9871      1.765          3        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.0it/s 1:16
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.3it/s 3.5s
                   all        232        235      0.698      0.757      0.669
0.232

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       4/20      3.85G      1.112     0.9207       1.71          4        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.0it/s 1:17
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 3.7it/s 4.0s
                   all        232        235      0.787      0.809      0.725
0.296

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
```

```
       5/20      3.85G       1.06     0.8897      1.679          2        640:
100% ━━━━━━━━━━━━ 305/305 4.0it/s 1:16
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.2it/s 3.5s
                   all        232        235      0.673      0.745      0.605
0.228

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       6/20      3.85G      1.054     0.8659      1.662          4        640:
100% ━━━━━━━━━━━━ 305/305 4.0it/s 1:17
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.2it/s 3.5s
                   all        232        235      0.768      0.717      0.727
0.316

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       7/20      3.85G      1.028     0.8317      1.643          4        640:
100% ━━━━━━━━━━━━ 305/305 4.0it/s 1:16
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.0it/s 3.7s
                   all        232        235      0.831      0.813      0.797
0.352

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       8/20      3.85G     0.9897     0.8396      1.613          2        640:
100% ━━━━━━━━━━━━ 305/305 4.0it/s 1:16
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.6it/s 3.2s
                   all        232        235      0.775      0.748       0.73
0.285

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
       9/20      3.85G     0.9637     0.8057        1.6          4        640:
100% ━━━━━━━━━━━━ 305/305 3.9it/s 1:17
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.0it/s 3.8s
                   all        232        235      0.803      0.781      0.744
0.316

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      10/20      3.85G     0.9469     0.7902      1.578          2        640:
100% ━━━━━━━━━━━━ 305/305 3.9it/s 1:18
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.3it/s 3.5s
                   all        232        235      0.813      0.762      0.809
0.401
Closing dataloader mosaic
albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01,
blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average',
num_output_channels=3), CLAHE(p=0.01, clip_limit=(1.0, 4.0), tile_grid_size=(8,
8))

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      11/20      3.85G     0.9696     0.5856      1.891          1        640:
100% ━━━━━━━━━━━━ 305/305 4.0it/s 1:15
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.1it/s 3.6s
                   all        232        235      0.831      0.858      0.831
0.395

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      12/20      3.85G     0.9094     0.5224       1.84          1        640:
100% ━━━━━━━━━━━━ 305/305 4.0it/s 1:16
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━ 15/15 4.4it/s 3.4s
```

```
                   all        232        235      0.809      0.868      0.842
0.42

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      13/20      3.85G      0.891     0.5053      1.805          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:15
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.0it/s 3.8s
                   all        232        235      0.806      0.834      0.851
0.428

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      14/20      3.85G     0.8969     0.4994      1.801          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:15
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.2it/s 3.5s
                   all        232        235      0.816      0.826      0.824
0.397

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      15/20      3.85G     0.8602     0.4803      1.745          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:15
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.2it/s 3.6s
                   all        232        235      0.828      0.826      0.815
0.364

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      16/20      3.85G     0.8577     0.4711      1.745          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:15
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.2it/s 3.6s
                   all        232        235      0.888      0.868       0.89
0.436

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      17/20      3.85G     0.8178     0.4708      1.712          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:14
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.2it/s 3.6s
                   all        232        235      0.869      0.851      0.891
0.471

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      18/20      3.85G     0.8043     0.4621      1.687          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:14
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.1it/s 3.7s
                   all        232        235      0.867      0.891      0.915
0.477

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      19/20      3.85G      0.769     0.4449      1.644          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:15
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.2it/s 3.6s
                   all        232        235      0.894      0.881      0.902
0.506

      Epoch    GPU_mem   box_loss   cls_loss   dfl_loss  Instances       Size
      20/20      3.85G     0.7535     0.4363      1.619          1        640:
100% ━━━━━━━━━━━━━━━━━━━━ 305/305 4.1it/s 1:15
                 Class     Images  Instances      Box(P          R      mAP50
mAP50-95): 100% ━━━━━━━━━━━━━━━━━━━━ 15/15 4.5it/s 3.3s
```

```
                 all        232        235      0.895      0.911      0.931
0.518

20 epochs completed in 0.448 hours.
```

эксперимент	кадров	уникальных ID	средняя длина трека (кадров)	коротких треков (<15 кадров)	объектов на кадр	максимум одновременно	доля кадров с треками	FPS полного прохода	
0	BoT-SORT: база	600	11	74.45	0	1.365	3	0.953	35.27
1	ByteTrack: база	600	11	89.45	1	1.640	3	0.987	43.13
2	ByteTrack: buffer=10	600	14	65.57	2	1.530	3	0.987	43.34
3	ByteTrack: buffer=90	600	10	98.50	1	1.642	3	0.988	44.15
4	ByteTrack: high=0.5	600	3	170.67	0	0.853	1	0.853	44.57
5	ByteTrack: match=0.5	600	89	5.83	83	0.865	4	0.615	44.64
6	ByteTrack: match=0.95	600	5	176.80	1	1.473	3	0.998	44.86
7	ByteTrack: conf=0.25	600	10	71.90	3	1.198	3	0.943	45.26
8	BoT-SORT: buffer=10	600	11	74.45	1	1.365	3	0.953	36.18
9	BoT-SORT: buffer=90	600	11	74.55	0	1.367	3	0.953	35.59

эксперимент	кадров	уникальных ID	средняя длина трека (кадров)	коротких треков (<15 кадров)	объектов на кадр	максимум одновременно	доля кадров с треками	FPS полного прохода
10	BoT-SORT: без GMC	600	9	102.89	1	1.543	3	0.985	43.94
11	BoT-SORT:	600	12	59.08	2	1.182	3	0.923	36.02

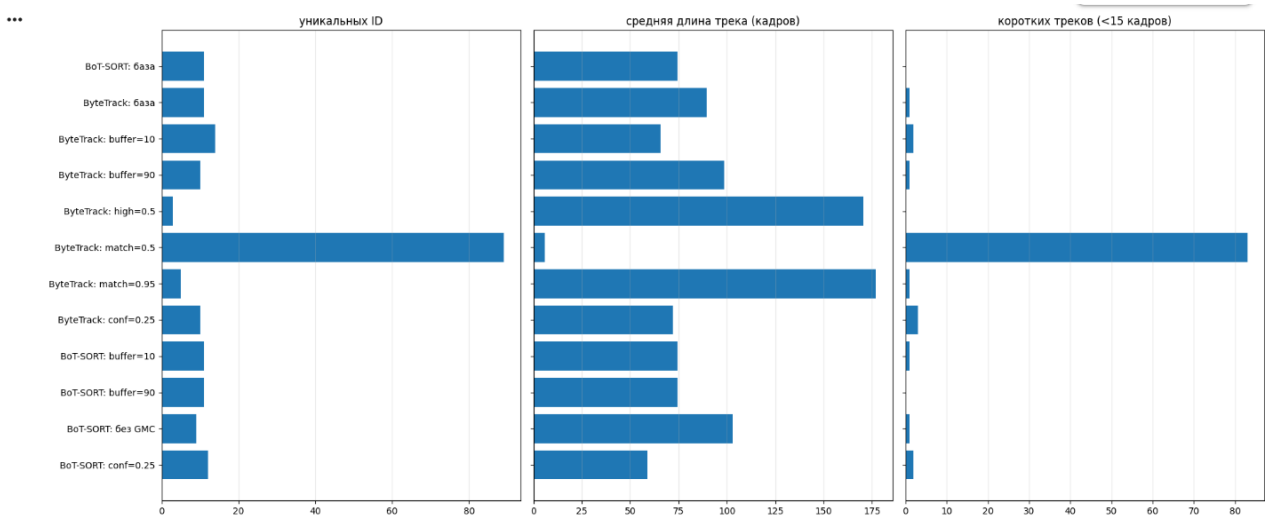
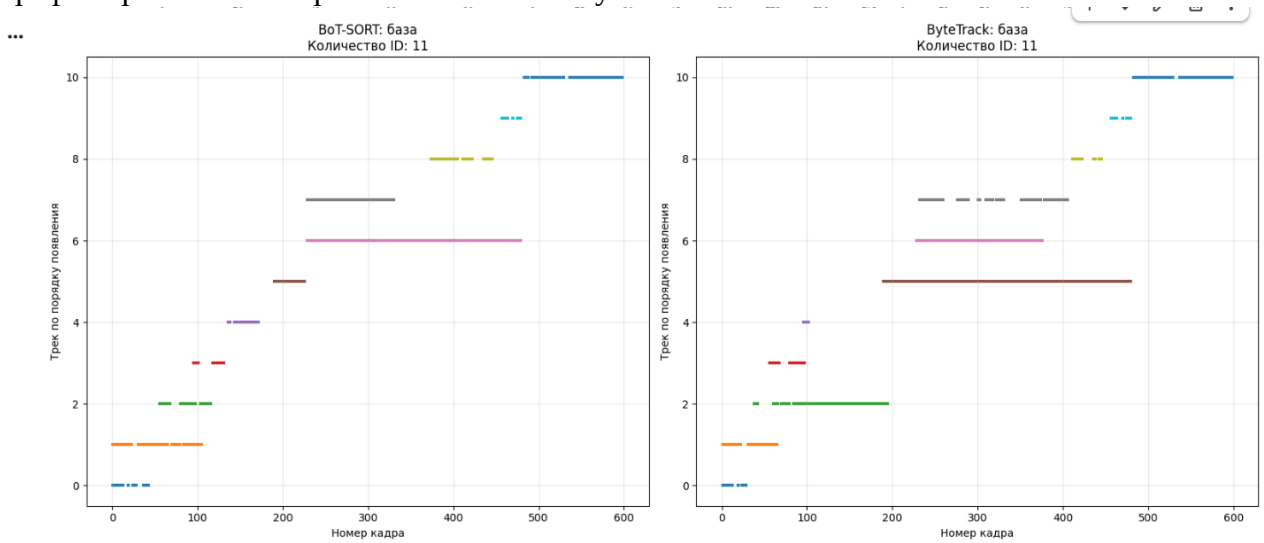


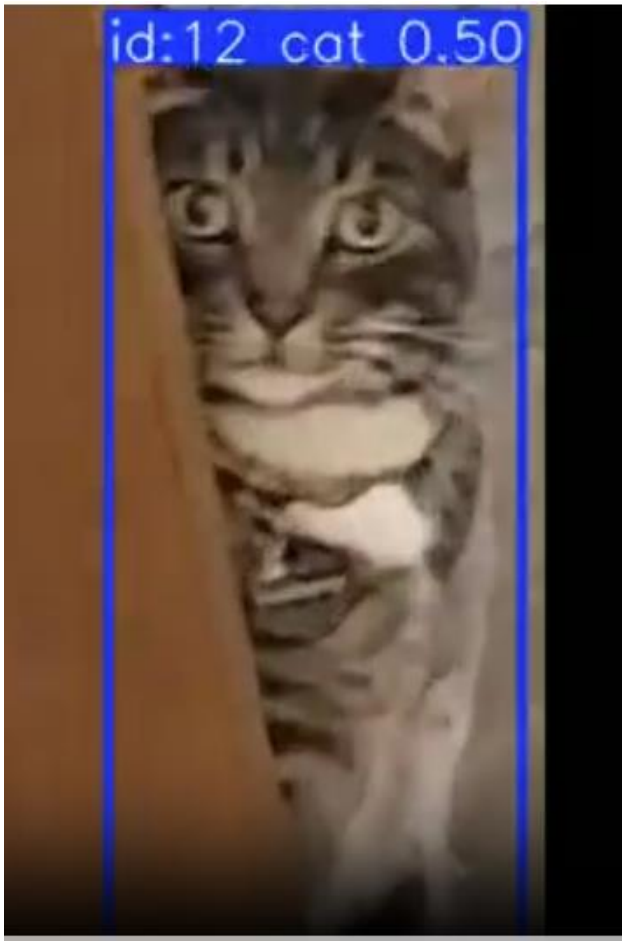
График сравнение алгоритмов BoT-Sort и ByteTrack:



Тут сразу видно, как каждая полоска показывает один ID от 1-го до последнего кадра, много коротких полосок, значит что трекер теряет объект и заведет себе новый ID.

Теперь продемонстрирую как распознается нужный мне объект на видео:







Вывод: Проделав данную лабораторную работу я исследовал применение алгоритмов трекинга на базе обученной сети-детектора объектов